LLMs from First Principles

Chapter 1: Before the Transformer

From counting words to learning where to look, 1948 to 2016

Samadeep Sengupta
samadeep.github.io/study

Draft 1, October 2026

Every number in this chapter is taken from the cited paper, with the table or section it came from.
Where something is reconstructed or illustrative, it is labelled as such.

Contents

1 Before the Transformer **2**

How to read this chapter 2

1 What a language model is 4

1.1 The object: a distribution over sequences 4

1.2 How we measure it: cross-entropy and perplexity 5

1.3 The one gradient you must know 6

1.4 Generation is a loop 6

2 Counting: n-gram models (1948 to 2013) 7

2.1 The Markov cut 7

2.2 Why counting cannot scale: the curse of dimensionality 7

2.3 Smoothing, and why Kneser-Ney was the bar to beat 8

3 Words become vectors: the neural probabilistic LM (2003) 9

3.1 The model, symbol by symbol 9

3.2 Why it generalises 9

4 Unbounded memory: recurrent networks (1990 to 2013) 11

4.1 One state to carry the whole past 11

4.2 Why gradients vanish or explode 12

4.3 Gradient clipping: the patch for explosion 13

5 Gated memory: LSTM and GRU (1997 to 2014) 13

5.1 The idea: an additive path 13

5.2 Equations 14

5.3 The GRU: two gates instead of three 15

6 Embeddings at scale: word2vec (2013) 16

6.1 Counting the cost 16

6.2 Negative sampling 17

7 Sequence to sequence (2013 to 2014) 18

7.1 From language models to conditional language models 18

7.2 Sutskever et al. (2014), counted 19

7.3 Decoding: greedy and beam search 20

8 Learning where to look: attention (2014 to 2015) 21

8.1 Bahdanau, Cho and Bengio: align and translate jointly 21

8.2 Luong, Pham and Manning: simpler, and the dot product 23

8.3 From these equations to queries, keys and values 24

9 Two loose ends, then the endgame 25

9.1 Subwords: fixing the vocabulary itself (2016) 25

9.2 The peak of the recurrent era (2016) 25

9.3 Why the transformer won: path length and parallelism 26

9.4 What you carry into Chapter 2 27

9.5 Blog cuts from this chapter 27

10 Exercises 28

References 30

Chapter 1

Before the Transformer

How to read this chapter

Every modern LLM, from GPT-2 to DeepSeek-V3, is trained on one objective: given the tokens so far, predict the next one. That objective is not new. It was written down by Claude Shannon in 1948 and attacked with counting, then with small neural networks, then with recurrent networks, then with attention. Each attempt fixed the failure of the one before it and exposed a new failure. The transformer (Chapter 2) is the fix for the last failure in this chain.

So this chapter is a ladder. Each rung is: *what problem was there*, *what the paper did*, *what broke next*. Read it in order. Figure 1.1 is the whole chapter on one page; come back to it whenever you lose the thread.



Figure 1.1: The chapter as a ladder. Each rung fixes the break (**-X-**) of the rung above it. The final break, sequential computation, is what the transformer removes.

> **General rule.** *A model architecture is a bet about which dependencies matter and how cheaply they can be computed. Every step up this ladder either (a) lets information travel further with less loss, or (b) lets more of the computation run at the same time. The transformer wins on both. Keep asking these two questions of every model in this book, including the MoE models of Chapter 6.*

What you should be able to do after this chapter

1. Define a language model as a probability distribution, derive cross-entropy and perplexity, and convert between them and bits per token.
2. Build a smoothed n-gram model by hand and explain exactly why it cannot generalise.
3. Count the parameters and FLOPs of Bengio's 2003 model and of Sutskever's 2014 LSTM, and match the published totals (384M for the latter).
4. Derive the vanishing-gradient bound for an RNN and explain why the LSTM cell state fixes it.

5. Write down Bahdanau and Luong attention, and map each symbol onto the query, key and value of Chapter 2.

6. Explain why the RNN lost: not because it modelled language worse, but because it could not use a GPU well.

Notation used throughout the book

Symbol	Meaning
$\mathcal{V}$, $\lvert\mathcal{V}\rvert$	vocabulary and its size
$w_{1:T}$	token sequence $w_1, \dots, w_T$
$x_t \in \mathbb{R}^d$	input vector at step t (usually an embedding)
$h_t \in \mathbb{R}^n$	hidden state at step t
W, U, V	weight matrices; shapes stated where they are introduced
$\sigma(z) = 1/(1+e^{-z})$	logistic sigmoid; $\sigma'(z) \le 1/4$
$\tanh$	hyperbolic tangent; $\tanh'(z) \le 1$
$\odot$	elementwise (Hadamard) product
$\mathrm{softmax}(z)_i = e^{z_i} / \sum_j e^{z_j}$	normalises a score vector into probabilities
$\log$	natural log unless written $\log_2$

Labels used in this chapter

Key idea. *The single idea a section exists to teach.*

Systems note. What the idea costs in compute, memory or communication, and which later chapter it turns into. You care about inference and datacenters, so these boxes are the thread that connects this chapter to Chapters 3 to 7.

Worked example. Real numbers from the paper, pushed through the formulas so you can check them. ◇

Where it breaks. The failure that motivates the next section, marked **-X-**, plus the guards that should have caught it and why they did not.

1 What a language model is

1.1 The object: a distribution over sequences

A language model (LM) assigns a probability to every finite sequence of tokens $w_{1:T}$ from a vocabulary $\mathcal{V}$. Nothing more. The trick that makes this tractable is the chain rule of probability, which is exact, not an approximation:

$$P(w_{1:T}) \;=\; \prod_{t=1}^{T} P(w_t \mid w_{1:t-1}). \tag{1.1}$$

So a model of sequences is the same thing as a model of *the next token given the past*. Every model in this book, the 2003 MLP, the 2014 LSTM, GPT-4-class transformers, implements exactly the right-hand side. They differ only in how they compress the history $w_{1:t-1}$ into something a function can consume.

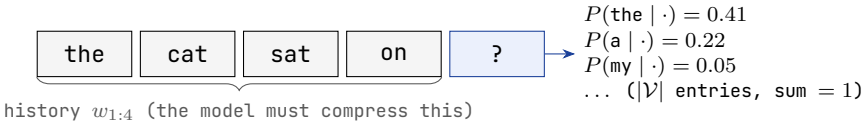


Figure 1.2: One factor of Eq. (1.1). The probabilities are illustrative, not from a model.

Paper. *Shannon, "A Mathematical Theory of Communication" (1948); "Prediction and Entropy of Printed English" (1951).* Shannon framed text as a stochastic process and generated synthetic English by sampling from letter and word n-gram statistics, showing the output looks more English as n grows. In 1951 he asked humans to guess the next letter of real text and bounded the entropy of English at roughly 0.6 to 1.3 bits per letter when long context is available. That experiment is literally a human language model evaluated by next-symbol prediction.

1.2 How we measure it: cross-entropy and perplexity

Let p be the true distribution of text and q_θ our model. The quantity we can estimate from a held-out corpus of N tokens is the average negative log-likelihood:

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{t=1}^{N} \log q_\theta(w_t \mid w_{1:t-1}). \tag{1.2}$$

This is a Monte Carlo estimate of the *cross-entropy* $H(p, q_\theta) = -\mathbb{E}_{w\sim p}[\log q_\theta(w)]$. Cross-entropy decomposes as

$$H(p, q) = H(p) + D_{\mathrm{KL}}(p \,\|\, q), \tag{1.3}$$

and since $D_{\mathrm{KL}} \geq 0$ with equality iff $q = p$, minimising $\mathcal{L}$ is the same as pulling the model toward the true distribution. The floor is the entropy of language itself, $H(p)$, which no model can beat. Shannon's 0.6 to 1.3 bits per letter is a human estimate of that floor.

Perplexity is just the exponent of the cross-entropy:

$$\mathrm{PPL} = \exp(\mathcal{L}) \qquad \text{(or } 2^{\mathcal{L}_2} \text{ if } \mathcal{L}_2 \text{ is in bits)}. \tag{1.4}$$

Interpretation: a model with perplexity k is, on average, as uncertain as if it were choosing uniformly among k tokens. A model that predicts uniformly over $|\mathcal{V}|$ has $\mathcal{L} = \log|\mathcal{V}|$ and $\mathrm{PPL} = |\mathcal{V}|$, the worst sensible score.

Worked example.

- Bengio et al. (2003) report test perplexity 109 on AP News. That is $\log_2 109 \approx 6.77$ bits per word.
- The Brown corpus result was 252, i.e. $\log_2 252 \approx 7.98$ bits per word. Smaller corpus, harder to model.
- Jozefowicz et al. (2016) reach 30.0 on the One Billion Word benchmark: $\ln 30 \approx 3.40$ nats, ≈ 4.91 bits per word.
- When a modern training curve reports loss 2.0 (nats per token), the per-token perplexity is $e^2 \approx 7.39$.

Perplexities are only comparable with the same vocabulary and tokenisation. Per-word and per-subword-token perplexities differ because the number of prediction events N differs. This is why papers after 2016 often report bits per byte or per character. ◇

1.3 The one gradient you must know

Almost every model here ends in a linear layer producing logits $z \in \mathbb{R}^{|\mathcal{V}|}$, then a softmax, then the log-loss of the correct index y. Write $p = \mathrm{softmax}(z)$ and $\ell = -\log p_y$. Then

$$\ell = -z_y + \log \sum_j e^{z_j}, \tag{1.5}$$

$$\frac{\partial \ell}{\partial z_i} = -\mathbb{1}[i = y] + \frac{e^{z_i}}{\sum_j e^{z_j}} \;=\; p_i - \mathbb{1}[i = y]. \tag{1.6}$$

So the gradient at the logits is "predicted distribution minus one-hot truth". It is bounded in $[-1, 1]$ per coordinate, which is part of why this loss trains stably. If the output layer is $z = W_o h$ with $W_o \in \mathbb{R}^{|\mathcal{V}| \times n}$, then $\partial \ell / \partial W_o = (p - e_y)\, h^\top$, a rank-one update touching *every* row of W_o.

Systems note. That last sentence is the cost centre of language modelling until about 2016. Computing p needs all $|\mathcal{V}|$ logits: $|\mathcal{V}| \cdot n$ multiply-adds per token. With $|\mathcal{V}| = 80{,}000$ and $n = 1000$ that is 80M multiply-adds per predicted token, more than all the LSTM layers combined in Sutskever's model (§7). Hierarchical softmax, importance sampling and subword vocabularies (§9.1) all exist to cut this term. In Chapter 4 you will see the same term reappear at inference as the lm_head matmul, and in Chapter 5 as a reason to shard the vocabulary across GPUs.

1.4 Generation is a loop

Eq. (1.1) also tells you how to generate: sample $w_1 \sim P(\cdot)$, then $w_2 \sim P(\cdot \mid w_1)$, and so on. Each step needs the previous step's output. During *training* the true prefix is known, so all positions can in principle be scored together ("teacher forcing"). During *generation* they cannot. This asymmetry is permanent. It is why LLM inference splits into a parallel *prefill* over the prompt and a serial *decode* loop (Chapter 4). Whether training can also be parallel over positions depends on the architecture, and that is the question this whole chapter builds toward.

Key idea. *A language model is a next-token distribution. Its quality is cross-entropy (equivalently perplexity) on held-out text. Its dominant per-token cost, before 2017, was the $|\mathcal{V}|$-way softmax. Its generation is inherently sequential.*

2 Counting: n-gram models (1948 to 2013)

2.1 The Markov cut

The history $w_{1:t-1}$ in Eq. (1.1) grows without bound. The oldest fix is to throw most of it away: assume the next token depends only on the last $n-1$ tokens.

$$P(w_t \mid w_{1:t-1}) \approx P(w_t \mid w_{t-n+1:t-1}). \tag{1.7}$$

This is an $(n-1)$-th order Markov assumption. With it, the maximum-likelihood estimate is a ratio of counts from a corpus:

$$\hat{P}_{\mathrm{ML}}(w \mid v) = \frac{c(v, w)}{c(v)}, \qquad v = w_{t-n+1:t-1}. \tag{1.8}$$

Worked example: a three-sentence corpus. Corpus: the cat sat . the dog sat . the cat ran .

bigram	count	$\hat{P}_{\mathrm{ML}}$
the cat	2	$2/3$
the dog	1	$1/3$
cat sat	1	$1/2$
cat ran	1	$1/2$
dog sat	1	1
dog ran	0	**0**

The perfectly reasonable test sentence the dog ran . gets probability $\frac{1}{3} \cdot 0 = 0$, so its log-likelihood is $-\infty$ and the test perplexity is infinite. One unseen bigram destroys the whole evaluation. ◇

2.2 Why counting cannot scale: the curse of dimensionality

The number of possible contexts is $|\mathcal{V}|^{n-1}$. Bengio et al. (2003, §1) make this concrete: modelling the joint distribution of 10 consecutive words over a 100,000-word vocabulary has potentially $100{,}000^{10} - 1 = 10^{50} - 1$ free parameters. No corpus covers more than a vanishing fraction of these contexts, so most test contexts are either unseen or seen a handful of times.

Where it breaks: the n-gram has no notion of similarity. -X- Having seen the cat sat a thousand times tells the model nothing about the dog sat. cat and dog are unrelated integer IDs. Every statistic is stored per exact string.

Guards that exist and why they do not fix it:

- *Smoothing* (below) moves probability mass to unseen events, but spreads it according to frequency, not meaning.
- *Backoff* to shorter contexts discards exactly the long-range information you wanted.
- *Class-based models* (Brown et al., 1992) cluster words by hand-crafted or distributional criteria, which helps, but the classes are hard assignments fixed before training, not learned jointly with the predictor.

The fix requires representing words as points in a space where distance means something. That is §3.

2.3 Smoothing, and why Kneser-Ney was the bar to beat

Three families, each a patch on the zero-count problem:

Add-one (Laplace). $\hat{P}(w \mid v) = \dfrac{c(v,w)+1}{c(v)+|\mathcal{V}|}$. Simple and terrible: with a 50k vocabulary it hands most of the mass to unseen events.

Interpolation. Mix orders: $\hat{P}(w \mid v) = \lambda\,\hat{P}_{\mathrm{ML}}(w \mid v) + (1-\lambda)\,\hat{P}(w \mid v')$ where v' is v with its oldest word dropped. Weights are tuned on held-out data.

Kneser-Ney (1995). Two ideas. First, *absolute discounting*: subtract a fixed $D \in (0,1)$ from every seen count and give the freed mass to the lower order. Second, and the clever part, the lower order should not use how often w occurs, but *how many different contexts* it completes. For a bigram model (interpolated form):

$$P_{\mathrm{KN}}(w \mid v) = \frac{\max(c(v,w) - D,\, 0)}{c(v)} + \lambda(v)\, P_{\mathrm{cont}}(w), \tag{1.9}$$

$$\lambda(v) = \frac{D \cdot N_{1+}(v\,\bullet)}{c(v)}, \qquad P_{\mathrm{cont}}(w) = \frac{N_{1+}(\bullet\, w)}{N_{1+}(\bullet\,\bullet)}, \tag{1.10}$$

where $N_{1+}(v\,\bullet)$ is the number of distinct words seen after v, and $N_{1+}(\bullet\, w)$ the number of distinct words seen before w. The standard illustration: "Francisco" may be frequent, but almost always after "San", so it has a tiny continuation count and should get little backoff mass in a new context. $\lambda(v)$ is exactly the mass removed by discounting, so the distribution still sums to one.

Chen and Goodman's (1998) empirical study made *modified* Kneser-Ney (three discounts, for counts 1, 2, and 3+) the standard baseline. Every neural LM paper in this chapter compares against it.

Worked example: how far counting got.

- AP News, 14M training words: the best n-gram in Bengio et al. (2003) is modified Kneser-Ney of order 5, test perplexity **117**.
- One Billion Word benchmark (Chelba et al., 2013): interpolated KN 5-gram with 1.1B n-grams, test perplexity **67.6**, about 1.76B parameters (as tabulated by Jozefowicz et al. 2016, Table 1).
- Brants et al. (2007) trained n-gram models on up to trillions of tokens for machine translation. Counting scales in data almost for free; it just stops getting better per byte.

◇

Systems note. An n-gram model is a giant hash table. Inference is a few lookups per token: no matrix multiply, no GPU. Its cost is memory (billions of entries) and its quality ceiling is set by the Markov cut. Keep this picture: when Chapter 4 describes the KV cache as "memory that grows with context", it is the learned, dense descendant of this table, and it hits the same wall, memory capacity, from the other side.

> **General rule.** *Counting estimates each context independently, so its sample efficiency is fixed by how often each exact string appears. Any model that wants to generalise must share statistical strength between contexts, which means it needs a representation in which "similar" contexts are near each other.*

3 Words become vectors: the neural probabilistic LM (2003)

Paper. *Bengio, Ducharme, Vincent, Jauvin, "A Neural Probabilistic Language Model", JMLR 3:1137–1155, 2003.* Learn, jointly: (1) a real-valued feature vector for every word, and (2) a neural network that maps the feature vectors of the previous $n-1$ words to a distribution over the next word. Because the network is a smooth function of the vectors, a sentence made of words *similar* to ones seen in training gets similar probability. The paper calls this fighting the curse of dimensionality with distributed representations.

3.1 The model, symbol by symbol

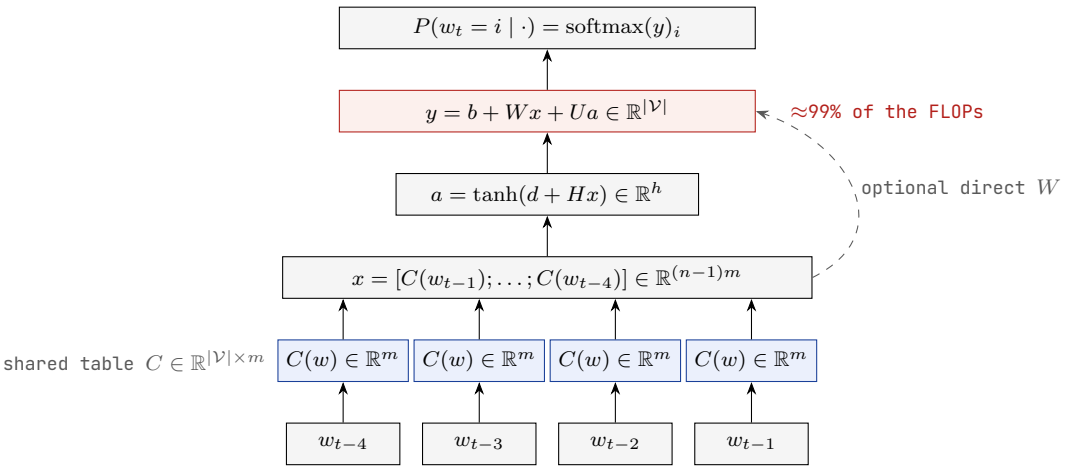


Figure 1.3: Bengio et al. (2003), Eq. (1) of the paper, drawn for $n = 5$ (four context words). C is shared across positions. The red layer is the one that costs.

The paper's model, with shapes:

$$x = \big(C(w_{t-1}),\, C(w_{t-2}),\, \ldots,\, C(w_{t-n+1})\big) \qquad \in \mathbb{R}^{(n-1)m} \tag{1.11}$$

$$y = b + Wx + U\tanh(d + Hx) \qquad \in \mathbb{R}^{|\mathcal{V}|} \tag{1.12}$$

$$\hat{P}(w_t = i \mid w_{t-n+1:t-1}) = e^{y_i} \big/ \textstyle\sum_j e^{y_j} \tag{1.13}$$

with $C \in \mathbb{R}^{|\mathcal{V}| \times m}$, $H \in \mathbb{R}^{h \times (n-1)m}$, $U \in \mathbb{R}^{|\mathcal{V}| \times h}$, $W \in \mathbb{R}^{|\mathcal{V}| \times (n-1)m}$ (or zero), biases $b \in \mathbb{R}^{|\mathcal{V}|}$, $d \in \mathbb{R}^h$. The paper gives the free-parameter count as

$$|\mathcal{V}|\,(1 + nm + h) \;+\; h\,\big(1 + (n-1)m\big), \tag{1.14}$$

dominated by $|\mathcal{V}|(nm + h)$: the embedding table, the direct connections and the output layer all scale with the vocabulary.

The embedding lookup is a matrix multiply in disguise. If e_i is the one-hot vector for word i, then $C(i) = C^\top e_i$. Nobody computes it that way (it is a row gather), but this view matters: gradients flow into C exactly as into any weight matrix, only into the rows that were used.

3.2 Why it generalises

Suppose cat and dog end up with nearby vectors because they appear in similar contexts. Then for the unseen the dog ran, the input x is close to the seen the

cat ran, and a smooth f gives a close output. The n-gram had to see each string; the neural model only has to see each *region* of embedding space. The paper reports learned features where semantically and syntactically similar words are close, which is the first appearance of what word2vec (§6) later makes famous.

Worked example: the 2003 results and the cost of the softmax. **Results** (paper, Tables 1 and 2):

Corpus	Train words	$\lvert\mathcal{V}\rvert$	best n-gram (test PPL)	MLP (test PPL)
Brown	800,000	16,383	312 (class-based, 500 classes)	**252** (-24%)
AP News	13,994,528	17,964	117 (modified KN, order 5)	**109** (-8%)

Parameter and FLOP count for the Brown MLP10 configuration as read from Table 1 of the paper ($n = 5$, $h = 100$, $m = 30$, no direct connections; it was also mixed with a trigram):

- C: $16{,}383 \times 30 = 491{,}490$
- H, d: $100 \times (4 \cdot 30) + 100 = 12{,}100$
- U, b: $16{,}383 \times 100 + 16{,}383 = 1{,}654{,}683$
- Total ≈ 2.16M parameters.

Per predicted word, the hidden layer costs $12{,}000$ multiply-adds; the output layer costs $1{,}638{,}300$. The output layer is **99.3%** of the arithmetic. The "neural" part is tiny; the vocabulary is the model. ◇

Systems note: the first vocabulary-parallel softmax (2003). Training on AP News took about three weeks for 5 epochs on 40 CPUs (paper, §4). The cluster was 32 dual-CPU 1.2 GHz Athlon nodes on Myrinet, programmed with MPI. Because the output layer dominates, the authors split it by *vocabulary*: CPU i owns a contiguous block of output units, every CPU redundantly computes the cheap embedding and hidden layers, and an MPI Allreduce combines the per-CPU partial sums of e^{y_j} to normalise the softmax (later with a shared max for numerical stability), plus the gradients flowing back into the hidden layer. Communication was about 1/15 of epoch time.

Read that again with Chapter 5 in mind. It is the same design Megatron-LM uses in 2019 for its vocabulary-parallel embedding and cross-entropy: shard the $\lvert\mathcal{V}\rvert$ dimension, all-reduce the max and the sum. The collective (all-reduce), the reason (the vocab-sized matmul does not fit or is too slow on one device) and the trick (never materialise the full logit vector on one device) are identical. Sixteen years apart.

Where it breaks: the fixed window. **-X-** The NPLM still conditions on exactly $n-1$ words. Increasing n grows H and x linearly, which is fine, but the model has a hard horizon: word $t-n$ cannot influence the prediction at all, no matter how relevant. The paper notes the model can be extended with recurrence. That is the next rung.

> **General rule.** *Replacing discrete symbols with learned dense vectors turns "have I seen this exact context?" into "is this context near ones I have seen?". Generalisation becomes a property of geometry. The price is that the output layer, a dense map back to $\lvert\mathcal{V}\rvert$ classes, now dominates compute and has to be parallelised along the vocabulary axis.*

4 Unbounded memory: recurrent networks (1990 to 2013)

4.1 One state to carry the whole past

A recurrent neural network (RNN) replaces the fixed window with a hidden state h_t updated once per token:

$$h_t = \phi\big(W_{hx}\, x_t + W_{hh}\, h_{t-1} + b\big), \qquad P(w_{t+1} \mid w_{1:t}) = \mathrm{softmax}(W_o\, h_t), \tag{1.15}$$

with ϕ a pointwise nonlinearity ($\tanh$ or σ). The same weights are reused at every step. In principle h_t depends on every token since the start, so the Markov cut is gone. Elman (1990) used this architecture to find structure in sequences; Rumelhart, Hinton and Williams (1986) and Werbos (1990) had given the training algorithm, backpropagation through time (BPTT): unroll the recurrence into a deep feedforward net with tied weights and run ordinary backprop.

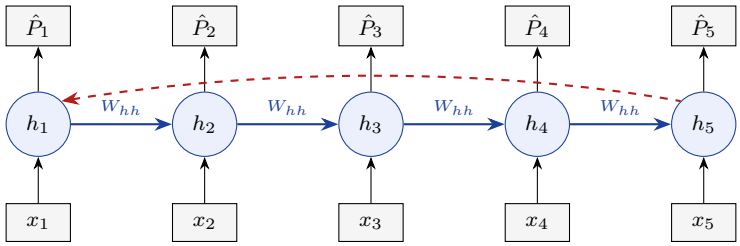


Figure 1.4: An RNN unrolled over five steps. Forward: information flows right along the teal edges. Backward: the error at step 5 reaches step 1 only through the same edges, multiplied by one Jacobian per step.

Paper. *Mikolov, Karafiát, Burget, Černocký, Khudanpur, "Recurrent neural network based language model", Interspeech 2010.* The input is the current word (one-hot) concatenated with the previous state; the state uses a sigmoid; the output is a softmax over the vocabulary. In the paper's notation $x(t) = [w(t); s(t-1)]$, $s_j(t) = f(\sum_i x_i(t)\, u_{ji})$, $y_k(t) = g\Big(\sum_j s_j(t)\, v_{kj}\Big)$. Hidden sizes of 30 to 500 are typical, larger for more data.

Worked example: RNNLM vs Kneser-Ney on WSJ (Mikolov 2010, Table 2). RNNs trained on 6.4M words of WSJ; the KN5 baseline is trained on the same data.

Model	PPL alone	PPL interpolated with KN5
KN5 baseline	221	n/a
single RNN (250 hidden)	173	155
3 RNNs, static mixture	151	143
3 RNNs, dynamic mixture	128	121

"Dynamic" means the model keeps training on the test stream as it reads it. The abstract's headline is roughly 50% lower perplexity for a mixture of RNN LMs versus a state-of-the-art backoff model, about 18% lower word error rate on WSJ speech recognition, and about 5% lower on NIST RT05 even though the backoff model there had far more data. ◇

Mikolov et al. (2013, word2vec paper, Eq. 3) give the per-token training complexity of an RNNLM as

$$Q_{\mathrm{RNN}} = H \times H + H \times V, \tag{1.16}$$

where H is the state size and V the vocabulary. Again the $H \times V$ softmax dominates unless replaced by a hierarchical softmax ($H \times \log_2 V$).

Systems note: the serial chain. Look at Eq. (1.15) as a schedule. To compute h_t you need h_{t-1}. A sequence of length T is therefore T *dependent* matrix-vector products with W_{hh}. You can batch across *different sequences* (making it matrix-matrix), but never across *time* within one sequence. On a GPU, a small matrix-vector product reads all of W_{hh} from memory to do only $2H^2$ FLOPs: it is memory-bandwidth bound, not compute bound. Chapter 3 formalises this with the roofline model, and Chapter 4 shows the transformer decoder hits the very same wall at inference time. The RNN hits it during *training* too. That is the real reason it lost.

4.2 Why gradients vanish or explode

Following Pascanu, Mikolov and Bengio (2013), use the form $h_t = W_{rec}\,\sigma(h_{t-1}) + W_{in}x_t + b$ (equivalent to the usual one up to where the nonlinearity sits). The loss is $E = \sum_t E_t$. By the chain rule,

$$\frac{\partial E_t}{\partial \theta} = \sum_{k \le t} \frac{\partial E_t}{\partial h_t}\,\frac{\partial h_t}{\partial h_k}\,\frac{\partial^+ h_k}{\partial \theta}, \qquad \frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}} = \prod_{i=k+1}^{t} W_{rec}^{\top}\,\mathrm{diag}\big(\sigma'(h_{i-1})\big). \tag{1.17}$$

The term $\partial h_t / \partial h_k$ carries error from step t back to step k, and it is a product of $t-k$ matrices.

Vanishing (sufficient condition). Let $|\sigma'| \le \gamma$, so $\|\mathrm{diag}(\sigma')\| \le \gamma$. Let λ_1 be the largest singular value (spectral norm) of W_{rec}.[1] If $\lambda_1 < 1/\gamma$ then every factor has norm at most $\eta = \lambda_1 \gamma < 1$, so

$$\left\| \frac{\partial E_t}{\partial h_t}\frac{\partial h_t}{\partial h_k} \right\| \le \eta^{\,t-k} \left\| \frac{\partial E_t}{\partial h_t} \right\| \xrightarrow{\;t-k\to\infty\;} 0 \text{ exponentially.} \tag{1.18}$$

For $\tanh$, $\gamma = 1$; for the sigmoid, $\gamma = 1/4$. With a sigmoid, even λ_1 slightly below 4 guarantees vanishing.

Exploding (necessary condition). Reversing the argument, for long-term components to grow, $\lambda_1 > 1/\gamma$ is necessary. In the linear case (σ = identity) the paper shows via power iteration that the error grows like λ_j^{t-k} along the leading eigenvector whenever $|\lambda_j| > 1$.

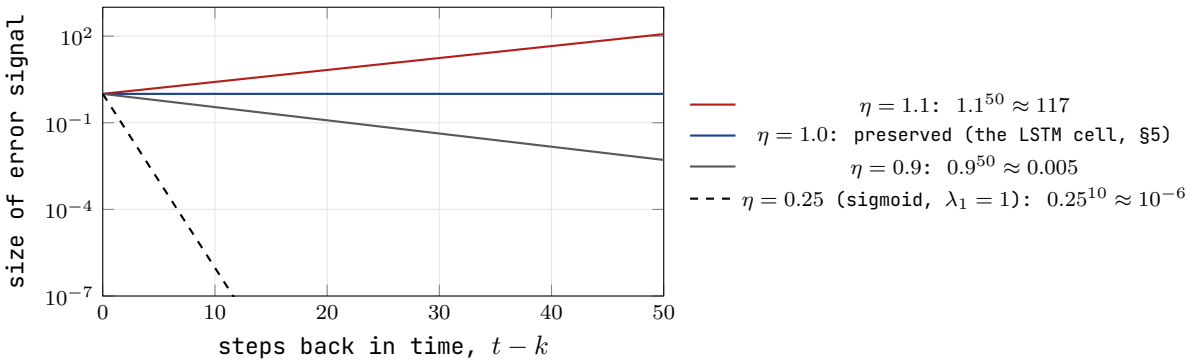


Figure 1.5: Error-signal magnitude after $t-k$ steps for a constant per-step factor η. These are the bounds from Eq. (1.17), not measurements. Real Jacobians vary per step, but the exponent is the point.

[1] Pascanu et al. phrase the condition with the absolute value of the largest eigenvalue. The norm bound in their proof, $\|W_{rec}^{\top}\|\,\|\mathrm{diag}(\sigma')\|$, actually needs the spectral norm; the two coincide for normal matrices, and the eigenvalue version is exact in their linear power-iteration analysis.

Where it breaks: long-range credit assignment. -X- The task needs word k to influence the loss at step $t \gg k$, but the gradient connecting them has shrunk by η^{t-k}. Learning that dependency is drowned out by short-range gradients. Bengio, Simard and Frasconi (1994) proved this difficulty for gradient descent, building on Hochreiter's 1991 diploma thesis.

Guards and why they failed:

- *Careful initialisation* ($\lambda_1 \approx 1$): holds only at step zero; training moves W_{rec}.
- *L1/L2 penalties* keep $\lambda_1 < 1$, which prevents explosion but, as Pascanu et al. note, forces the network into a regime where all information dies out exponentially. It trades exploding for vanishing.
- *Echo state networks* fix W_{rec} with spectral radius below 1 and do not learn it, so they also forget exponentially.
- *Gradient clipping* (below) stops explosion, but does nothing for vanishing.

The real fix changes the architecture so that a path exists where $\eta = 1$ by construction. That is §5.

4.3 Gradient clipping: the patch for explosion

Pascanu et al. (2013, Algorithm 1) rescale the whole gradient when its norm crosses a threshold:

$$\hat{g} \leftarrow \nabla_\theta E; \qquad \text{if } \|\hat{g}\| \geq \tau: \;\; \hat{g} \leftarrow \frac{\tau}{\|\hat{g}\|}\,\hat{g}. \tag{1.19}$$

Geometric story from the paper: an exploding direction creates a steep "wall" in the loss surface; a full gradient step at the wall throws parameters far away; a clipped step keeps the direction but bounds the jump. Mikolov's earlier recipe clipped elementwise; norm clipping preserves the direction. Sutskever et al. (2014) clip at $\tau = 5$; Jozefowicz et al. (2016) at $\tau = 1$. Modern LLM training still clips global gradient norm, typically at 1.0. Same algorithm.

> **General rule.** *Backprop through a recurrence multiplies the error by one Jacobian per step, so its size after L steps behaves like η^L. Any architecture that wants long memory must either make some path have $\eta \equiv 1$ (gating, §5; residual connections, Chapter 2) or make the path length L itself short (attention, §8). Clipping only stops the $\eta > 1$ side from blowing up a single update.*

5 Gated memory: LSTM and GRU (1997 to 2014)

5.1 The idea: an additive path

The vanishing gradient comes from *multiplying* the state by a matrix and squashing it at every step. Hochreiter and Schmidhuber (1997) introduced a memory cell whose self-connection is fixed at weight 1, the "constant error carousel": error flowing back along it is neither scaled up nor down. Gates, small sigmoid layers, learn when to write into the cell and when to read from it. The original 1997

cell had input and output gates only. The *forget gate*, which lets the cell reset itself, was added by Gers, Schmidhuber and Cummins (2000), and that version is what everyone now calls "the LSTM".

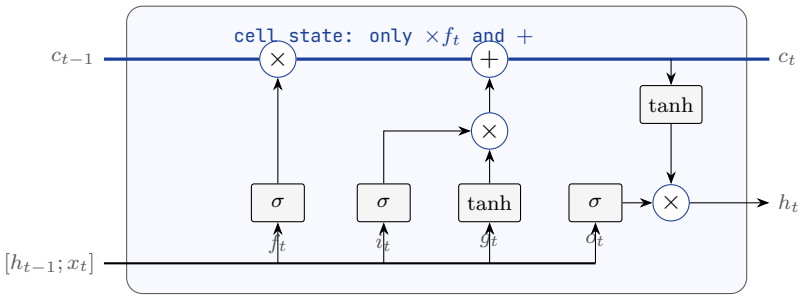


Figure 1.6: The LSTM cell (with forget gate). The top line is the cell state. The only operations on it are an elementwise multiply by f_t and an add, which is why error travels along it without being squashed.

5.2 Equations

For input $x_t \in \mathbb{R}^d$ and state size n:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \qquad \text{input gate} \tag{1.20}$$
$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \qquad \text{forget gate} \tag{1.21}$$
$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \qquad \text{output gate} \tag{1.22}$$
$$g_t = \tanh(W_g x_t + U_g h_{t-1} + b_g) \qquad \text{candidate} \tag{1.23}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \qquad \text{cell update (additive)} \tag{1.24}$$
$$h_t = o_t \odot \tanh(c_t) \qquad \text{exposed state} \tag{1.25}$$

The gradient path. Along the cell line, holding the gates fixed,

$$\frac{\partial c_t}{\partial c_{t-1}} = \operatorname{diag}(f_t) \quad \Longrightarrow \quad \frac{\partial c_t}{\partial c_k} = \prod_{i=k+1}^{t} \operatorname{diag}(f_i). \tag{1.26}$$

There is no weight matrix and no squashing derivative in this product. When the network sets $f_i \approx 1$ ("remember"), the factor is ≈ 1 and the error survives. This is the $\eta = 1$ curve in Figure 1.5, and the network *chooses* it per unit, per step. A standard trick is to initialise $b_f = 1$ so the cell starts out remembering; Jozefowicz et al. (2016) use exactly this.

Parameter count. Four gates, each with an $n \times d$ input matrix, an $n \times n$ recurrent matrix, and a bias:

$$P_{\text{LSTM}} = 4\left(nd + n^2 + n\right). \tag{1.27}$$

Worked example: checking one layer of Sutskever's LSTM. Sutskever et al. (2014) use $n = d = 1000$. One layer: $4(10^6 + 10^6 + 10^3) = 8{,}004{,}000 \approx 8$M parameters. Four layers ≈ 32M. The paper says the model has 64M "pure recurrent connections", 32M for the encoder LSTM and 32M for the decoder. Matches. We finish this count in §7. ◇

5.3 The GRU: two gates instead of three

Cho et al. (2014) proposed a simpler gated unit while introducing the encoder-decoder (§7). There is no separate cell; the state itself is interpolated. In the paper's notation, per unit j:

$$r_j = \sigma\big([W_r x]_j + [U_r h_{t-1}]_j\big) \qquad \text{reset gate} \tag{1.28}$$

$$z_j = \sigma\big([W_z x]_j + [U_z h_{t-1}]_j\big) \qquad \text{update gate} \tag{1.29}$$

$$\tilde{h}_j = \tanh\big([W x]_j + [U(r \odot h_{t-1})]_j\big) \tag{1.30}$$

$$h_{t,j} = z_j\, h_{t-1,j} + (1 - z_j)\, \tilde{h}_j \tag{1.31}$$

When $z_j \approx 1$ the unit copies its previous value, the same additive escape route as the LSTM cell. When $r_j \approx 0$ the candidate ignores the past, letting a unit drop information. The paper notes that units learning short-term dependencies tend to have active reset gates, and long-term ones active update gates. Parameter count: $3(nd + n^2 + n)$, i.e. three quarters of an LSTM of the same width. Cho et al. also report they could not get meaningful results with a plain $\tanh$ unit without gating.

Convention warning: Bahdanau et al. (2014) write the same unit as $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$, with z swapped. Same model, different label.

Which is better? Later empirical comparisons (for example Jozefowicz, Zaremba and Sutskever, 2015) did not find either to dominate across tasks; a forget-gate bias of 1 closes much of the LSTM's gap. In practice LSTMs dominated large-scale LM and translation until 2017.

Systems note: what an LSTM step costs. Per token per layer, the four gates are one fused matmul of $[x_t; h_{t-1}] \in \mathbb{R}^{d+n}$ against a $(d+n) \times 4n$ matrix: $4n(d+n)$ multiply-adds. Optimised GPU implementations (described by Appleyard, Kočiský and Blunsom, 2016, and shipped in cuDNN) precompute the Wx_t half for *all* time steps in one big parallel matmul, because inputs are known in advance; only the Uh_{t-1} half must run serially. That half is still a small matmul repeated T times, bounded by memory bandwidth unless the batch is large.

The numbers from that paper make the point. A naive LSTM forward pass (hidden 512, minibatch 64) on an NVIDIA M40 ran at about **0.4 TFLOPS** against a peak of about **5.8 TFLOPS**: under 10% utilisation. Six optimisations (grouping the four gate GEMMs, running the input and recurrent GEMMs on separate CUDA streams, fusing the pointwise ops into one kernel, pre-transposing weights, batching inputs over time, and overlapping layers in a diagonal wavefront) gave an 11.1× speedup on a four-layer network. Every one of those tricks is about finding parallelism the recurrence hides. Gating fixed the optimisation problem; it did not touch the scheduling problem.

Where it breaks: still a chain. **-X-** LSTMs made long-range learning possible, but two costs remain. (1) The path from token k to token t is still $t - k$ steps long, so information must survive $t - k$ gated updates; Khandelwal et al. (2018) measured that LSTM language models make use of roughly 200 tokens of context, and that word order matters only within the most recent 50 tokens (local order within 20); further back, the model keeps a rough "topic" of which words appeared. (2) The computation is serial in t. A GPU with tens of thousands of arithmetic lanes sits mostly idle on a batch of short matrix-vector steps.

General rule. *Make the state update additive and gate it with learned values in $[0, 1]$; the backward pass then contains a product of gate values instead of a product of weight matrices, and the network can learn to keep it near 1. The same principle, "give the gradient an identity path", reappears as the residual connection $x + f(x)$ in every transformer block (Chapter 2).*

6 Embeddings at scale: word2vec (2013)

By 2012 everyone agreed word vectors were useful, but training them meant training a whole language model, and the expensive parts (the nonlinear hidden layer and the $|\mathcal{V}|$-way softmax) capped corpora at a few hundred million words. Mikolov, Chen, Corrado and Dean asked the opposite question: what is the *cheapest* model that still produces good vectors?

Paper. *Mikolov, Chen, Corrado, Dean, "Efficient Estimation of Word Representations in Vector Space", arXiv:1301.3781, 2013.* Remove the hidden layer. Predict a word from the average of its neighbours' vectors (CBOW), or predict the neighbours from the word (Skip-gram). Replace the softmax by a Huffman-coded hierarchical softmax. Train on billions of words in about a day.

6.1 Counting the cost

The paper defines training complexity as $O = E \times T \times Q$ (epochs $\times$ training words $\times$ per-example cost) and gives Q per architecture (its Eqs. 2 to 5), with context size N, vector size D, hidden size H, vocabulary V, window C:

Model	Q per training example	dominant term
Feedforward NNLM	$ND + NDH + HV$	NDH (with hier. softmax)
RNNLM	$HH + HV$	HH (with hier. softmax)
CBOW	$ND + D \log_2 V$	tiny
Skip-gram	$C\,(D + D \log_2 V)$	tiny

With a binary tree over the vocabulary, a softmax over V classes becomes $\log_2 V$ binary decisions: about 20 instead of 10^6 for a million-word vocabulary. A Huffman tree gives frequent words shorter codes, roughly halving that again.

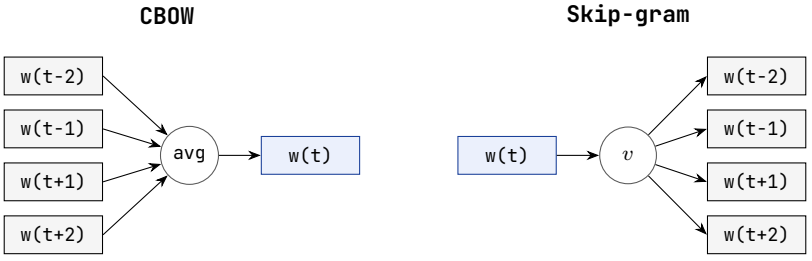


Figure 1.7: The two log-linear architectures (after Figure 1 of Mikolov et al. 2013a). No hidden nonlinearity: the model is a dot product between an input vector and an output vector, followed by a (hierarchical) softmax.

6.2 Negative sampling

The follow-up paper (Mikolov, Sutskever, Chen, Corrado, Dean, NIPS 2013, "Distributed Representations of Words and Phrases and their Compositionality") replaced the hierarchical softmax with *negative sampling*, a simplification of noise-contrastive estimation. For an input word w_I and an observed context word w_O, maximise

$$\log \sigma\big({v'_{w_O}}^{\top} v_{w_I}\big) + \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n(w)} \Big[\log \sigma\big(- {v'_{w_i}}^{\top} v_{w_I}\big) \Big], \tag{1.32}$$

i.e. classify the true pair as real and k random pairs as fake. The noise distribution that worked best was the unigram distribution raised to the $3/4$ power, $P_n(w) \propto U(w)^{3/4}$, which boosts rare words relative to their raw frequency. The paper recommends k of 5 to 20 for small corpora and 2 to 5 for large ones, and subsamples very frequent words. Per training pair the cost is now $(k+1)$ dot products of length D, independent of $|\mathcal{V}|$.

What it is secretly computing. Levy and Goldberg (2014) showed that skip-gram with negative sampling implicitly factorises a word-context matrix whose entries are the pointwise mutual information $\mathrm{PMI}(w,c) = \log \frac{P(w,c)}{P(w)P(c)}$ shifted by $\log k$. So word2vec is, at heart, a fast low-rank factorisation of co-occurrence statistics: the bridge between counting (§2) and neural models (§3).

Worked example: what the cheap model bought (Mikolov et al. 2013a). Evaluation is an analogy test: 8,869 semantic and 10,675 syntactic questions of the form "big is to biggest as small is to ?", answered by the nearest vector to $v_{\texttt{biggest}} - v_{\texttt{big}} + v_{\texttt{small}}$; only an exact match counts.

Same data, 640-dim vectors (Table 3):

	semantic	syntactic
RNNLM	9%	36%
NNLM	23%	53%
CBOW	24%	64%
Skip-gram	55%	59%

Scaled up on 6B words of Google News, DistBelief (Table 6):

Model	dim	total accuracy	training (days $\times$ CPU cores)
NNLM	100	50.8%	14×180
CBOW	1000	63.7%	2×140
Skip-gram	1000	65.6%	2.5×125

Skip-gram gets higher accuracy with 10$\times$ larger vectors at about one eighth of the CPU-days. Table 2 of the paper also shows accuracy saturating if you grow only data or only dimension; you must grow both. That is a scaling law in miniature, eight years before Kaplan et al. (Chapter 2). ◇

Systems note: the parameter server. These models were trained on DistBelief (Dean et al., 2012): 50 to 100 model replicas each reading a shard of the data, sending gradients to and pulling parameters from a central sharded parameter server, with asynchronous mini-batch SGD and Adagrad. This is data parallelism with *asynchronous* updates. Chapter 5 explains why large-model training moved to *synchronous* data parallelism with an all-reduce (ring or tree) instead: stale gradients hurt convergence for big dense models, and GPU interconnects made the collective cheap. The vocabulary of distributed training, replica, shard, staleness, starts here.

Where it breaks: one vector per word. -X- word2vec gives bank one vector, whether the sentence is about rivers or money. The vector is a frequency-weighted blur of all senses. Guard that existed: train separate vectors per sense (multi-prototype embeddings, e.g. Huang et al. 2012); it needs the number of senses fixed in advance and does not condition on the actual sentence. The real fix is to compute a word's representation *from its context* every time it is used. An RNN's hidden state already does that; an encoder that reads the whole sentence does it better. That is where we go next. (Contextual embeddings from LSTMs, ELMo in 2018, and from transformers, BERT in 2018, are the endpoint.)

> **General rule.** *When the representation, not the predictor, is what you need, strip the model to the smallest thing that still has to solve a prediction problem, and spend the savings on data. Training signal comes from co-occurrence; the model is a factorisation of it. Every input embedding table in an LLM still plays this role, but is learned jointly with a deep predictor that turns it into context-dependent vectors.*

7 Sequence to sequence (2013 to 2014)

7.1 From language models to conditional language models

Machine translation needs $P(y_{1:T'} \mid x_{1:T})$: a language model over the target sentence, *conditioned* on the source. Apply the chain rule to the target only:

$$P(y_{1:T'} \mid x) = \prod_{t=1}^{T'} P(y_t \mid y_{1:t-1}, x). \tag{1.33}$$

The 2013 to 2014 papers all realise this with two networks: an *encoder* that reads x and produces a representation, and a *decoder*, an RNN language model that also sees that representation. Kalchbrenner and Blunsom (2013) were first to map a whole sentence to a vector and back (with a convolutional encoder); Cho et al. (2014) and Sutskever, Vinyals and Le (2014) did it with gated RNNs on both sides.

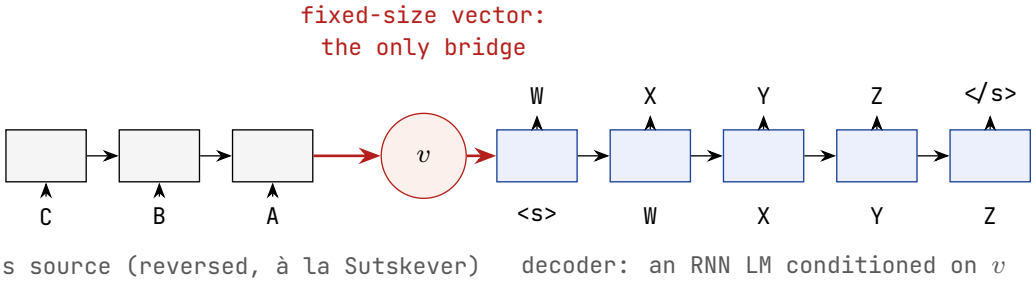


Figure 1.8: The encoder-decoder (after Figure 1 of Sutskever et al. 2014). Everything the decoder knows about the source has to pass through v.

Paper. *Cho, van Merriënboer, Gulcehre, Bahdanau, Bougares, Schwenk, Bengio, "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation", EMNLP 2014.* Introduces the RNN Encoder-Decoder and the GRU (§5). The encoder's final state is mapped to a summary $c = \tanh(Vh_{\langle N \rangle})$; the decoder is

initialised from c *and* receives c at every step. 1000 hidden units, 15,000-word vocabularies on each side (covering about 93% of the data), trained on 348M words selected from 850M. Crucially, it is used to *score phrase pairs* as an extra feature inside the Moses phrase-based system (baseline 33.3 BLEU on newstest2014), not yet to translate on its own.

Paper. *Sutskever, Vinyals, Le, "Sequence to Sequence Learning with Neural Networks", NIPS 2014*. Two deep LSTMs (4 layers $\times$ 1000 cells), no phrase table, no SMT system. Read the source *reversed*. Decode with left-to-right beam search. First pure neural system to beat a phrase-based SMT baseline on a large task.

7.2 Sutskever et al. (2014), counted

Worked example: reproducing "384M parameters". Configuration (paper §3.4): 4 LSTM layers per side, $n = d = 1000$, input vocabulary 160,000, output vocabulary 80,000, naive softmax over 80k.

Component	Parameters
source embeddings $160{,}000 \times 1000$	160.0M
target embeddings $80{,}000 \times 1000$	80.0M
encoder LSTM, $4 \times 4(1000 \cdot 1000 + 1000^2 + 1000)$	32.0M
decoder LSTM, same	32.0M
output softmax $80{,}000 \times 1000$ (+ bias)	80.1M
total	**≈384M**

The paper states 384M total of which 64M are recurrent. Our count matches. Note where the parameters are: **83%** are in vocabulary-sized tables. The "sentence representation" is 4 layers $\times$ (h and c) $\times 1000 =$ **8,000 numbers**, which the paper also states.

Results on WMT'14 En→Fr, newstest2014 (Tables 1 and 2):

System	BLEU
Phrase-based SMT baseline (Schwenk)	33.30
Single forward LSTM, beam 12	26.17
Single *reversed* LSTM, beam 12	30.59
Ensemble of 5 reversed LSTMs, beam 1	33.00
Ensemble of 5 reversed LSTMs, beam 12	**34.81**
Rescoring SMT 1000-best with 5 LSTMs	36.5
Best WMT'14 system (as re-scored by the authors)	37.0

The reversal trick: test perplexity 5.8 $\rightarrow$ 4.7 and BLEU 25.9 $\rightarrow$ 30.6 (§3.3), from nothing but reading the source backwards. ◇

Why reversal works. The average distance between a source word and its translation is unchanged, but the *first* source words are now adjacent to the first target words. The paper's phrase: it reduces the "minimal time lag" so backprop can "establish communication" early. In the language of §4: it shortens the shortest gradient path from $L \approx T$ to $L \approx 1$ for the first few words, and the rest is bootstrapped from there. Hold onto that: *path length is what matters*. Attention will make it 1 for every word.

7.3 Decoding: greedy and beam search

Finding $\arg\max_y P(y \mid x)$ exactly is intractable ($|\mathcal{V}|^{T'}$ candidates). Beam search keeps the B highest-scoring prefixes; at each step it extends each by every word, scores by cumulative log-probability, and keeps the top B; a hypothesis that emits </s> moves to the finished set. $B = 1$ is greedy decoding. Sutskever et al. found $B = 2$ gives most of the benefit (Table 1: 33.00 at $B = 1$, 34.50 at $B = 2$, 34.81 at $B = 12$, for the 5-model ensemble). Because log-probabilities only decrease, plain beam search prefers short outputs; later systems divide by length (Sennrich et al. 2016 normalise by sentence length).

BLEU (Papineni et al., 2002), the metric in every table here: the geometric mean of modified (clipped) n-gram precisions for $n = 1..4$, times a brevity penalty $\mathrm{BP} = \min(1, e^{1-r/c})$ for candidate length c and reference length r. It rewards n-gram overlap with a reference, not meaning. Different tokenisation scripts give different BLEU for the same output; Sutskever et al. note that the best WMT'14 system scores 37.0 under their script but 35.8 as officially reported.

Systems note: the first big model-parallel LSTM. A single-GPU C++ implementation ran at about 1,700 words per second, too slow. So (paper §3.5) they used an **8-GPU machine**:

- **4 GPUs, one LSTM layer each.** Each GPU passes its activations to the next "as soon as they were computed". Layer 2 can process time step t while layer 1 processes $t + 1$: a diagonal wavefront. This is *pipeline parallelism* over layers (Chapter 5: GPipe, 2018).
- **4 GPUs for the softmax,** each multiplying by a $1000 \times 20{,}000$ slice of the $1000 \times 80{,}000$ output matrix. This is *vocabulary-sharded tensor parallelism*, the same split Bengio used on CPUs in 2003 (§3).

Result: 6,300 words per second at minibatch 128, about 10 days per model. They also bucketed sentences by length so a minibatch is not mostly padding, which alone gave a 2× speedup. Padding waste in batches is exactly what continuous batching (Chapter 4) attacks at inference time.

Where it breaks: the fixed-size bottleneck. **-X-** Every source sentence, 5 words or 80, is squeezed into the same 8,000 numbers (Sutskever) or 1,000 (Cho). Cho et al.'s follow-up analysis ("On the Properties of Neural Machine Translation", 2014) found the basic encoder-decoder degrades quickly as sentences get longer. Sutskever et al. were surprised that their deep, reversed LSTM did *not* degrade much below 35 words, but it took a 384M-parameter ensemble, reversal and a huge state to get there.

Guards and why they were not enough:

- *Bigger state*: more numbers in v, but the decoder still reads the same vector at every step regardless of which word it is translating.
- *Reversal*: helps the beginning of the sentence and, empirically, the rest; but it is a data trick, not a mechanism, and does not help when reordering is long-range (for example German verbs at the end).
- *Feeding c at every decoder step* (Cho): the decoder sees the summary repeatedly, but it is still one summary.

What the decoder actually needs, when producing target word i, is the *part* of the source that word i is about. Let it look.

General rule. *A conditional language model must route information from the input to every output position. If all of it goes through one fixed-size vector, capacity and gradient path length are both fixed by that vector, independent of input length. Expanding the bridge helps only linearly; replacing a single bridge with a per-output, content-addressed read is the structural fix.*

8 Learning where to look: attention (2014 to 2015)

8.1 Bahdanau, Cho and Bengio: align and translate jointly

Paper. *Bahdanau, Cho, Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate", arXiv:1409.0473 (2014), ICLR 2015.* Conjecture: the fixed-length vector is the bottleneck. Fix: keep *one vector per source word* and, for each target word, compute a fresh weighted average of them. The weights are produced by a small network and learned end to end. The paper calls it a soft search over the source; the decoder "decides parts of the source sentence to pay attention to."

Encoder. A bidirectional RNN (GRU units) reads the source forwards and backwards. The annotation of word j concatenates both directions, so it summarises the whole sentence with a focus around j:

$$h_j = \big[\overrightarrow{h}_j\,;\,\overleftarrow{h}_j\big] \in \mathbb{R}^{2n}. \tag{1.34}$$

Decoder. For target position i, with previous decoder state s_{i-1}:

$$e_{ij} = v_a^\top \tanh\big(W_a s_{i-1} + U_a h_j\big) \qquad \text{score} \tag{1.35}$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})} \qquad \text{weights, sum to 1} \tag{1.36}$$

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij}\, h_j \qquad \text{context for word } i \tag{1.37}$$

$$s_i = f(s_{i-1},\, y_{i-1},\, c_i), \qquad P(y_i \mid y_{<i}, x) = g(y_{i-1},\, s_i,\, c_i). \tag{1.38}$$

with $W_a \in \mathbb{R}^{n' \times n}$, $U_a \in \mathbb{R}^{n' \times 2n}$, $v_a \in \mathbb{R}^{n'}$. The paper's interpretation: α_{ij} is the probability that target word i is aligned to source word j, and c_i is the *expected annotation* under that distribution. Because everything is differentiable, the alignment is not a latent variable to be inferred (as in classical SMT); its gradient flows straight from the translation loss.

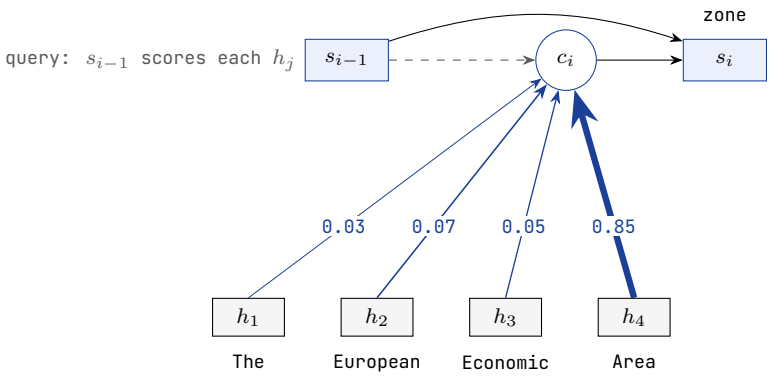


Figure 1.9: Producing the French word zone: the decoder state scores every source annotation; the context is their weighted sum. The weights shown are illustrative, chosen to match the behaviour the paper describes for *European Economic Area* → *zone économique européenne*, where the model aligns zone with Area first, skipping two words, then steps back.

Worked example: setup and results (Bahdanau et al. 2014, §4 to §5, Appendix A). **Data:** WMT'14 En→Fr, 850M parallel words reduced by data selection to 348M; 30,000-word shortlist per language, everything else [UNK]; test = news-test-2014, 3,003 sentences.
Sizes: $n = 1000$ hidden units per direction and in the decoder; embeddings $m = 620$; maxout layer $l = 500$; alignment MLP $n' = 1000$. Adadelta, minibatch 80, about 5 days per model.

Model	BLEU, all sentences	BLEU, no [UNK]
RNNencdec-30	13.93	24.19
RNNsearch-30	21.50	31.44
RNNencdec-50	17.82	26.71
RNNsearch-50	26.75	34.16
RNNsearch-50, trained longer	**28.45**	**36.15**
Moses (phrase-based, + 418M monolingual words)	33.30	35.63

(-30/-50 = trained on sentences up to 30/50 words.) On sentences without unknown words the attention model beats Moses while using no monolingual data. In the paper's Figure 2, RNNencdec's BLEU drops sharply as sentence length grows, while RNNsearch-50 shows no deterioration even past 50. Note that RNNsearch-**30** beats RNNencdec-**50**: attention trained on short sentences generalises better than no attention trained on long ones. ◇

Soft beats hard. The paper's example: the man → l' homme. A hard one-to-one alignment maps the→l', but choosing l' over le/la/les requires knowing the next word is man. Soft attention puts weight on both and gets it right. It also handles phrases of different lengths with no special NULL alignment.

Predecessor. Graves (2013) had used a differentiable attention window for handwriting synthesis, a mixture of Gaussians whose centre could only move forward. Bahdanau et al. point out that monotonicity is fatal for translation, which needs long-range reordering. Their attention is content-based and can jump anywhere.

Systems note: the first key cache. Appendix A.1.2 of the paper notes the alignment MLP must be evaluated $T_x \times T_y$ times per sentence pair, and that since $U_a h_j$ does not depend on i, "we can pre-compute it in advance to minimize the computational cost." Precompute the projection of every source position once, store it, and at each

decoding step only project the new query and compare. Change "source position" to "every previous token" and that is the **KV cache** of Chapter 4, which dominates GPU memory in LLM serving. The cost structure is already visible here: attention over T_x positions costs $O(T_x)$ per output token and $O(T_x T_y)$ per sentence. Bahdanau et al. flag the quadratic cost themselves as a possible limit for tasks longer than translation's 15 to 40 words. In 2026 that is the long-context problem.

8.2 Luong, Pham and Manning: simpler, and the dot product

Paper. *Luong, Pham, Manning, "Effective Approaches to Attention-based Neural Machine Translation", EMNLP 2015.* Systematises attention: *global* (attend to all source words) vs *local* (attend to a window), three *score functions*, and *input feeding*. Uses a stacked 4-layer LSTM and the decoder's current top-layer state h_t as the query (Bahdanau used the previous state s_{i-1}).

Computation path. $h_t \to a_t \to c_t \to \tilde{h}_t \to y_t$:

$$a_t(s) = \frac{\exp\big(\text{score}(h_t, \bar{h}_s)\big)}{\sum_{s'} \exp\big(\text{score}(h_t, \bar{h}_{s'})\big)}, \qquad c_t = \sum_s a_t(s)\, \bar{h}_s, \tag{1.39}$$

$$\tilde{h}_t = \tanh\big(W_c[c_t; h_t]\big), \qquad P(y_t \mid y_{<t}, x) = \text{softmax}(W_s \tilde{h}_t), \tag{1.40}$$

with three content-based scores:

$$\text{score}(h_t, \bar{h}_s) = \begin{cases} h_t^\top \bar{h}_s & \textit{dot} \\ h_t^\top W_a\, \bar{h}_s & \textit{general} \\ v_a^\top \tanh\big(W_a[h_t; \bar{h}_s]\big) & \textit{concat} \text{ (Bahdanau-style)} \end{cases} \tag{1.41}$$

Local attention (local-p). Predict an aligned position $p_t = S \cdot \sigma\big(v_p^\top \tanh(W_p h_t)\big) \in [0, S]$ for source length S, attend only within $[p_t - D,\, p_t + D]$ (they use $D = 10$), and multiply the weights by a Gaussian $\exp\big(-(s - p_t)^2 / 2\sigma^2\big)$ with $\sigma = D/2$. Fixed cost per target word regardless of source length, and differentiable almost everywhere, unlike "hard" attention that samples one position.

Input feeding. Concatenate $\tilde{h}_t$ into the input of the next step, so the decoder knows what it attended to before. A soft version of SMT's "coverage" bookkeeping.

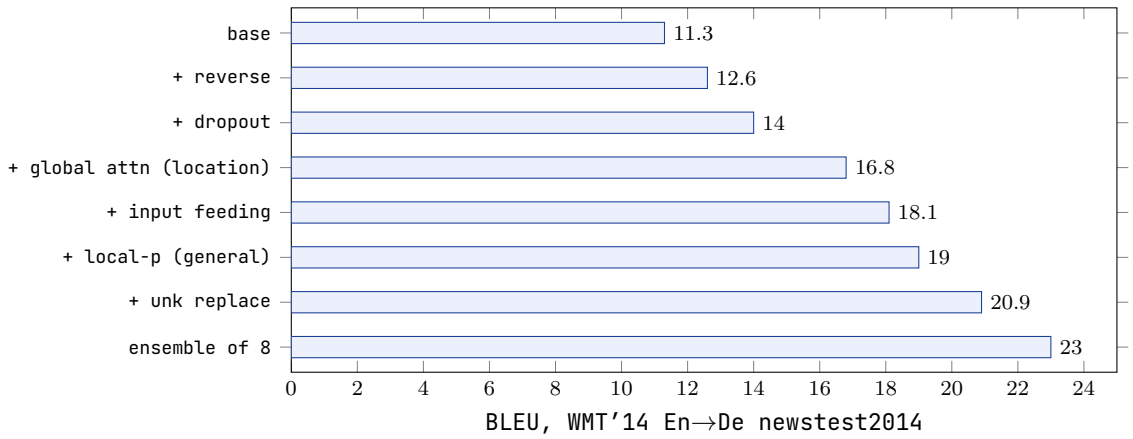


Figure 1.10: Luong et al. (2015), Table 1: each row adds one change. Attention plus input feeding plus local-p is +5.0 BLEU over the non-attentional model that already has reversal and dropout. The previous best NMT system (Jean et al. 2015) scored 21.6; the WMT'14 winner (phrase-based) 20.7.

Worked example: Luong et al. 2015. 4.5M sentence pairs (116M English, 110M German words), 50k vocabularies, 4-layer LSTM with 1000 cells and 1000-dim embeddings, plain SGD, batch 128, gradient norm clipped at 5, dropout 0.2. About 1K target words per second on one Tesla K40; 7 to 10 days per model. The ensemble set a new state of the art of **25.9 BLEU** on WMT'15 En→De, +1.0 over the best system. Among score functions, *dot* worked best for global attention and *general* for local; their *concat* implementation underperformed. ◇

8.3 From these equations to queries, keys and values

Rewrite the *general* score with $W_a = W_Q^\top W_K$ (any matrix factors this way, possibly with a low-rank bottleneck):

$$\text{score}(h_t, \bar{h}_s) = h_t^\top W_Q^\top W_K \, \bar{h}_s = (W_Q h_t)^\top (W_K \bar{h}_s) = q_t^\top k_s. \tag{1.42}$$

Now name things: the **query** q_t is "what this decoder step is looking for"; each **key** k_s is "what source position s advertises"; and the vectors averaged, here $\bar{h}_s$ itself, are the **values**. Bahdanau's $U_a h_j$ is a key projection; his $W_a s_{i-1}$ a query projection; his additive $\tanh$ combination is just a different similarity than the dot product.

	Query from	Keys / values from	Similarity
Bahdanau 2014	previous decoder state s_{i-1}	BiRNN encoder states	additive MLP
Luong 2015	current decoder state h_t	top encoder layer	dot / bilinear
Transformer 2017	*every* position	*every* position (self-attention)	scaled dot, many heads

One detail Chapter 2 will need: if q and k have d independent zero-mean unit-variance coordinates, then $\text{Var}(q^\top k) = d$. With $d = 512$, typical scores have standard deviation about 22, and the softmax saturates into a near one-hot, with near-zero gradients for every other position. Dividing by $\sqrt{d}$ restores unit variance. That is the $\sqrt{d_k}$ in "scaled dot-product attention".

Where it breaks: attention was still a passenger on an RNN. **-X-** Attention fixed the bridge between encoder and decoder: every target word now has a path of length 1 to every source word. But inside the encoder, two source words k apart are still connected only through k recurrent steps, and inside the decoder the same holds for target words. And training still walks the sequence one step at a time.

Guard that existed: *bidirectional* encoders let every annotation see both sides of the sentence. But information from word i still reaches word j only through $|i - j|$ recurrent steps, in whichever direction runs from i to j; for the first and last words that is $T - 1$ steps either way. The worst-case path is unchanged, and the serial work doubles.

The question Vaswani et al. asked in 2017: if attention gives path length 1 across the bridge, why not use it *within* each sequence too, and drop the recurrence entirely?

> **General rule.** *Attention is a differentiable, content-addressed lookup: score a query against every key, normalise with a softmax, return the weighted sum of values. It makes the gradient path between any two attended positions a single step, at a cost linear in the number of positions per query and quadratic per sequence. Everything after 2017 is a decision about where queries, keys and values come from, and how to pay for the quadratic term.*

9 Two loose ends, then the endgame

9.1 Subwords: fixing the vocabulary itself (2016)

Every model so far uses a word vocabulary with a cutoff: 16k (Bengio), 30k (Bahdanau), 50k (Luong), 80k to 160k (Sutskever). Every other word becomes [UNK]. Translation, though, is open-vocabulary: names, numbers, compounds like German *Abwasserbehandlungsanlage*. And the vocabulary size is exactly the term that dominates parameters and FLOPs (§1, §3, §7).

Paper. *Sennrich, Haddow, Birch, "Neural Machine Translation of Rare Words with Subword Units", ACL 2016*. Adapt byte pair encoding (Gage, 1994), a compression algorithm that repeatedly replaces the most frequent pair of bytes with a new symbol, to word segmentation. Start from characters, repeatedly merge the most frequent adjacent pair of symbols. The number of merges is the only hyperparameter, and it sets the vocabulary size. Frequent words end up as one symbol; rare words split into reusable pieces.

Worked example: BPE by hand and at scale. The paper's toy dictionary (Algorithm 1): low×5, lower×2, newest×6, widest×3, each split into characters plus an end-of-word marker. Running the paper's Python code verbatim, the first five merges are (e,s), (es,t), (est,</w>), (l,o), (lo,w), with pair counts 9, 9, 9, 7, 7 (for example e s occurs in newest×6 and widest×3). The unseen word lowest then segments into two known pieces, low + est</w>.

German side of WMT'15 (paper Table 1):

Segmentation	tokens	types (vocabulary)	unknowns in test
none (words)	100M	1,750,000	1,079
characters	550M	3,000	0
BPE (59,500 merges)	112M	63,000	0

BPE costs only 12% more tokens than words, with a vocabulary 28× smaller and zero unknowns. Characters have the smallest vocabulary, but 5.5× longer sequences, which, for a sequential RNN, means 5.5× more serial steps. ◇

Systems note. Tokenisation is a compute decision. Sequence length multiplies every per-token cost (and, for attention, squares one of them); vocabulary size sets the embedding table and the output matmul. BPE (and its descendants: WordPiece, SentencePiece, byte-level BPE) sits at the knee of that trade-off. A 2026 LLM vocabulary of roughly 100k to 250k subwords is the same idea with more merges and bytes as the base alphabet so nothing is ever unknown. When Chapter 4 talks about "tokens per second", the token is a BPE piece.

9.2 The peak of the recurrent era (2016)

Paper. *Jozefowicz, Vinyals, Schuster, Shazeer, Wu, "Exploring the Limits of Language Modeling", arXiv:1602.02410, 2016*. Large, regularised LSTM language models on the One Billion Word benchmark (about 0.8B training words, 793,471-word vocabulary). Importance-sampled softmax, character-CNN inputs, 32 Tesla K40 GPUs.

Worked example: how far LSTMs got (Jozefowicz et al. 2016, Tables 1 and 2).

Model	test PPL	params
Interpolated KN 5-gram, 1.1B n-grams	67.6	1.76B
Previous best single model (RNN + MaxEnt 9-gram features)	51.3	20B
2-layer LSTM-8192-1024 ("BIG LSTM")	30.6	1.8B
BIG LSTM + CNN character inputs	**30.0**	1.04B
Ensemble of 10 LSTMs + SNM10-skip n-gram	**23.7**	n/a

- Of the best model's 1.04B parameters, **820M** are the softmax projection. Even at the peak, the vocabulary is most of the model.
- The softmax over 793,471 words is approximated with 8,192 shared noise samples per step: a $(2560 \times 1024) \cdot (1024 \times 8193)$ matmul instead of $(2560 \times 1024) \cdot (1024 \times 793471)$, about 100× less compute.
- Best results after about **3 weeks** on 32 GPUs. Their observation: the best models were the largest that would fit in GPU memory. "Size matters."

◇

This paper is the bridge to everything that follows. Its conclusion, *bigger models trained on more data keep getting better*, is the scaling hypothesis. Its constraint, *a serial recurrence that cannot use the hardware*, is what the transformer removes. And one of its authors, Noam Shazeer, co-authors both the Transformer (2017) and the sparsely-gated Mixture-of-Experts layer (2017) that Chapter 6 starts from.

9.3 Why the transformer won: path length and parallelism

Put the three layer types side by side, for a sequence of length T and width d. This is the comparison Vaswani et al. (2017, Table 1) used to motivate the transformer; Chapter 2 derives each entry.

Layer	compute per layer	sequential steps	longest path between two positions
Recurrent	$O(T \cdot d^2)$	$O(T)$	$O(T)$
Convolutional (kernel k)	$O(k \cdot T \cdot d^2)$	$O(1)$	$O(\log_k T)$
Self-attention	$O(T^2 \cdot d)$	$O(1)$	$O(1)$

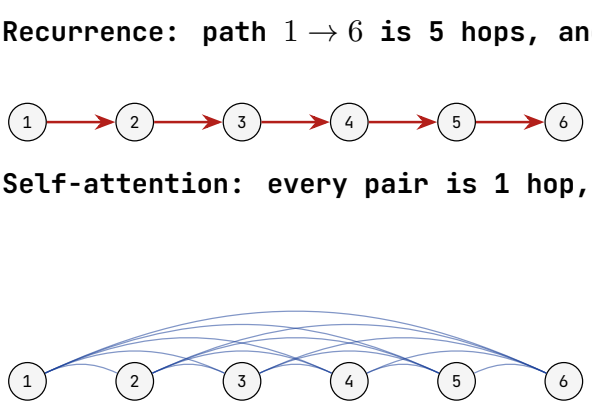


Figure 1.11: The two quantities this chapter has been tracking. Path length controls how far gradient and information must travel (§4, §7). Sequential steps control whether a GPU can work on all positions at once (§5 systems box). Self-attention makes both constant, and pays with T^2 pairwise scores.

When $T < d$, which was true for sentence-level translation ($T \approx 30$, $d \approx 1000$), self-attention is also *cheaper* per layer than recurrence. The T^2 term only bites at long contexts, and dealing with it (FlashAttention, KV-cache management, sparse and linear attention) fills much of Chapters 3 and 4.

Key idea. *The transformer did not win because recurrence models language badly; Jozefowicz's LSTMs were excellent. It won because it converts the same modelling job into large, independent matrix multiplies that saturate a GPU during training, and because it gives every pair of positions a one-hop path. The hardware and the gradients both prefer it.*

9.4 What you carry into Chapter 2

From this chapter	Becomes, in the transformer
chain rule, cross-entropy, perplexity (§1)	the training objective, unchanged
embedding table C (§3)	token embedding; often tied with the output layer
vocabulary-sharded softmax (§3, §7)	lm_head and vocab-parallel cross-entropy (Ch. 5)
gradient identity path, $f_t \approx 1$ (§5)	residual connections $x + F(x)$
encoder-decoder (§7)	original transformer; decoder-only LMs drop the encoder
attention, $q^\top k$ score, softmax, weighted sum of values (§8)	scaled dot-product, multi-head self-attention
precomputed $U_a h_j$ (§8)	KV cache (Ch. 4)
BPE (§9)	the tokenizer
beam search, greedy (§7)	decoding strategies; sampling with temperature
pipeline and vocab-sharded GPUs (§7)	pipeline and tensor parallelism (Ch. 5)

9.5 Blog cuts from this chapter

Each is one idea, with the evidence already in this chapter:

1. **"The vocabulary was the model."** 99% of Bengio 2003's FLOPs, 83% of Sutskever 2014's parameters, 79% of Jozefowicz 2016's parameters are vocabulary-sized tables. Why tokenisation is a compute decision.
2. **"Vocab-parallel softmax is from 2003."** Bengio's MPI all-reduce over output-unit shards, Sutskever's 4-GPU softmax, Megatron's parallel cross-entropy: the same diagram three times.
3. **"Reversing the input bought 4.7 BLEU."** Path length as the hidden variable behind reversal, gating, attention and residuals.
4. **"The first KV cache."** Bahdanau's one-line footnote on precomputing $U_a h_j$, and how it grew into the dominant memory cost of LLM serving.
5. **"RNNs lost on the GPU, not on the benchmark."** 0.4 of 5.8 TFLOPS; the 11× of tricks it took to claw back; why attention needed none of them.

10 Exercises

Do them with pen and paper first; a calculator is fine. Answers follow. The ones marked ⋆ are the ones that will matter most in Chapters 3 to 5.

E1. Perplexity by hand. A model assigns probabilities $1/2$, $1/4$, $1/8$ to the three correct tokens of a test sequence. Compute the average NLL in nats and bits, and the perplexity.

E2. Prove that a model predicting uniformly over $|\mathcal{V}|$ tokens has perplexity exactly $|\mathcal{V}|$, and that no model can have perplexity below $\exp(H(p))$ on data from p.

E3. Kneser-Ney. On the corpus of §2 (the cat sat . the dog sat . the cat ran ., treating . as a token), with $D = 0.75$, compute $P_{\mathrm{KN}}(\mathsf{ran} \mid \mathsf{dog})$ and $P_{\mathrm{KN}}(\mathsf{sat} \mid \mathsf{dog})$. Then show that $\sum_w P_{\mathrm{KN}}(w \mid \mathsf{dog}) = 1$.

E4. ⋆ Where the FLOPs are. For Bengio's AP News configuration ($|\mathcal{V}| = 17{,}964$, $n = 6$, $h = 60$, $m = 100$; assume no direct connections), count the parameters and the fraction of per-word multiply-adds spent in the output layer.

E5. Derive Eq. (1.6). Then show the gradient at the logits always sums to zero across the vocabulary, and say what that means for a model that is already certain and correct.

E6. Vanishing. A $\tanh$ RNN has $\|W_{rec}\|_2 = 0.95$. Bound the error signal after 100 steps. How many steps until the bound drops below 10^{-3}? For a sigmoid RNN with $\|W_{rec}\|_2 = 2$, what is the bound after 10 steps?

E7. Count the parameters of one GRU layer with $n = d = 1000$ and compare with the LSTM layer of §5.

E8. ⋆ Softmax vs recurrence. In Sutskever et al.'s decoder, compute per generated token the multiply-adds in (a) the four LSTM layers and (b) the 80k softmax. Which dominates, by how much?

E9. ⋆ The first key cache. In Bahdanau's model with $n = n' = 1000$ (so $U_a \in \mathbb{R}^{1000 \times 2000}$), translating a 30-word sentence into 30 words: how many multiply-adds go into computing $U_a h_j$ if you recompute it at every decoder step, versus once per source word? What is the ratio in general?

E10. Show that if $q, k \in \mathbb{R}^d$ have i.i.d. zero-mean, unit-variance coordinates, $\mathrm{Var}(q^\top k) = d$. What scaling makes the variance 1?

E11. BPE. Starting from characters with an end-of-word marker, on the word counts hug×10, pug×5, pun×12, bun×4, hugs×5: what is the first merge, with what count? What tie appears at the second merge, and why does tie-breaking matter for reproducibility?

E12. Path lengths. In a bidirectional RNN encoder over T words, what is the longest path (in recurrent steps) by which information from word i can reach the annotation of word j? Compare with a single self-attention layer.

E13. ⋆ Utilisation. One LSTM step with hidden size 512, input size 512, minibatch 64: count the FLOPs (2 per multiply-add) of the gate matmuls. Appleyard et al. report 777 μs per cell for the naive implementation on an M40 (peak ≈ 5.8 TFLOPS). What utilisation is that? What does it tell you about where the time went?

Answers

E1. NLL $= (\ln 2 + \ln 4 + \ln 8)/3 = 2\ln 2 \approx 1.386$ nats $= 2$ bits. Perplexity $= e^{2\ln 2} = 4$.

E2. Uniform: $\mathcal{L} = -\log(1/|\mathcal{V}|) = \log|\mathcal{V}|$, so $\mathrm{PPL} = |\mathcal{V}|$. Lower bound: $H(p,q) = H(p) + D_{\mathrm{KL}}(p\|q) \geq H(p)$, and $\exp$ is increasing.

E3. Distinct bigram types: (the,cat), (cat,sat), (sat,.), (.,the), (the,dog), (dog,sat), (cat,ran), (ran,.) so $N_{1+}(\bullet\bullet) = 8$. For context dog: $c(\texttt{dog}) = 1$, one distinct follower, so $\lambda(\texttt{dog}) = 0.75 \cdot 1/1 = 0.75$. $P_{\mathrm{cont}}(\texttt{ran}) = 1/8$ (only cat precedes it), $P_{\mathrm{cont}}(\texttt{sat}) = 2/8$. So $P(\texttt{ran} \mid \texttt{dog}) = 0 + 0.75/8 = 0.09375$ (no longer zero) and $P(\texttt{sat} \mid \texttt{dog}) = 0.25 + 0.75 \cdot 2/8 = 0.4375$. Sum: discounted mass 0.25 plus $0.75\sum_w P_{\mathrm{cont}}(w) = 0.75$, total 1.

E4. C: 1,796,400; H, d: $60 \times 500 + 60 = 30{,}060$; U, b: $1{,}077{,}840 + 17{,}964$. Total 2,922,264. Per word: hidden $30{,}000$ MACs, output $1{,}077{,}840$: **97.3%** in the output layer.

E5. See §1. $\sum_i (p_i - \mathbb{1}[i = y]) = 1 - 1 = 0$. If the model already puts $p_y \approx 1$, every component is ≈ 0: confident correct predictions stop contributing gradient. (This is also why label smoothing, which never lets the target be exactly one-hot, keeps a small gradient alive.)

E6. $0.95^{100} \approx 0.0059$. Need $0.95^L < 10^{-3}$: $L > \ln 10^{-3}/\ln 0.95 \approx 134.7$, so 135 steps. Sigmoid: $\eta = 2 \times \frac{1}{4} = 0.5$, $0.5^{10} \approx 9.8 \times 10^{-4}$.

E7. GRU: $3(10^6 + 10^6 + 10^3) = 6{,}003{,}000$, exactly $3/4$ of the LSTM's $8{,}004{,}000$.

E8. (a) $4 \times 4 \times 1000 \times (1000 + 1000) = 32$M MACs. (b) $80{,}000 \times 1000 = 80$M MACs. Softmax is $2.5\times$ all four LSTM layers. This is why they gave the softmax 4 of the 8 GPUs.

E9. One projection is $1000 \times 2000 = 2$M MACs. Recomputed every step: $T_x T_y \times 2\text{M} = 900 \times 2\text{M} = 1.8$G MACs. Cached: $T_x \times 2\text{M} = 60$M. Ratio $= T_y = 30$. In general caching saves a factor equal to the number of decoding steps, and costs memory $O(T_x \cdot n')$. Exactly the KV-cache trade-off of Chapter 4.

E10. $q^\top k = \sum_i q_i k_i$; terms are independent with $\mathbb{E}[q_i k_i] = 0$, $\mathrm{Var}(q_i k_i) = \mathbb{E}[q_i^2]\mathbb{E}[k_i^2] = 1$, so the variance is d. Divide by $\sqrt{d}$.

E11. Pair counts: (u,g) appears in hug, pug, hugs: $10 + 5 + 5 = 20$, the first merge. Second step: (u,n) and (n,</w>) both have count 16 (from pun×12 and bun×4). Different tie-breaks give different vocabularies and therefore different token IDs for the same text; a tokenizer is only reproducible if its tie-breaking (and merge list) is fixed and shipped with the model.

E12. $|i - j|$ steps, so up to $T - 1$; the backward direction does not shorten the path between two given positions, it only adds the other direction. One self-attention layer: 1 step for every pair.

E13. FLOPs $= 2 \times 64 \times (4 \cdot 512) \times (512 + 512) \approx 268$M. In 777$\mu$s that is ≈ 0.35 TFLOPS, about **6%** of 5.8. The arithmetic is not the bottleneck: kernel launches, separate small GEMMs that cannot fill all SMs, and round trips of pointwise intermediates to DRAM are. Chapter 3 names these: launch overhead, occupancy, and memory traffic.

References

Read in full for this chapter (numbers above are taken from these, with table or section given in the text): Bengio et al. 2003; Mikolov et al. 2010; Pascanu et al. 2013; Mikolov et al. 2013a; Cho et al. 2014a; Sutskever et al. 2014; Bahdanau et al. 2015; Luong et al. 2015; Sennrich et al. 2016; Jozefowicz et al. 2016; Appleyard et al. 2016; Khandelwal et al. 2018.

Cited for background (claims stated at the level of the paper's abstract or well-known result): the rest.

[1] Appleyard, J., Kočiský, T., Blunsom, P. Optimizing Performance of Recurrent Neural Networks on GPUs. arXiv:1604.01946, 2016.

[2] Bahdanau, D., Cho, K., Bengio, Y. Neural Machine Translation by Jointly Learning to Align and Translate. ICLR 2015; arXiv:1409.0473.

[3] Bengio, Y., Ducharme, R., Vincent, P., Jauvin, C. A Neural Probabilistic Language Model. JMLR 3:1137–1155, 2003.

[4] Bengio, Y., Simard, P., Frasconi, P. Learning Long-Term Dependencies with Gradient Descent is Difficult. IEEE Trans. Neural Networks 5(2):157–166, 1994.

[5] Brants, T., Popat, A., Xu, P., Och, F., Dean, J. Large Language Models in Machine Translation. EMNLP-CoNLL 2007.

[6] Brown, P., et al. Class-Based n-gram Models of Natural Language. Computational Linguistics 18(4), 1992.

[7] Chelba, C., et al. One Billion Word Benchmark for Measuring Progress in Statistical Language Modeling. arXiv:1312.3005, 2013.

[8] Chen, S., Goodman, J. An Empirical Study of Smoothing Techniques for Language Modeling. Harvard TR-10-98, 1998.

[9] Cho, K., et al. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. EMNLP 2014; arXiv:1406.1078. *(2014a)*

[10] Cho, K., van Merriënboer, B., Bahdanau, D., Bengio, Y. On the Properties of Neural Machine Translation: Encoder-Decoder Approaches. SSST-8, 2014. *(2014b)*

[11] Dean, J., et al. Large Scale Distributed Deep Networks. NIPS 2012.

[12] Elman, J. Finding Structure in Time. Cognitive Science 14:179–211, 1990.

[13] Gage, P. A New Algorithm for Data Compression. C Users Journal 12(2):23–38, 1994.

[14] Gers, F., Schmidhuber, J., Cummins, F. Learning to Forget: Continual Prediction with LSTM. Neural Computation 12(10):2451–2471, 2000.

[15] Graves, A. Generating Sequences with Recurrent Neural Networks. arXiv:1308.0850, 2013.

[16] Hochreiter, S. Untersuchungen zu dynamischen neuronalen Netzen. Diploma thesis, TU München, 1991.

[17] Hochreiter, S., Schmidhuber, J. Long Short-Term Memory. Neural Computation 9(8):1735–1780, 1997.

[18] Huang, E., Socher, R., Manning, C., Ng, A. Improving Word Representations via Global Context and Multiple Word Prototypes. ACL 2012.

[19] Huang, Y., et al. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. NeurIPS 2019; arXiv:1811.06965.

[20] Jean, S., Cho, K., Memisevic, R., Bengio, Y. On Using Very Large Target Vocabulary for Neural Machine Translation. ACL 2015.

[21] Jozefowicz, R., Zaremba, W., Sutskever, I. An Empirical Exploration of Recurrent Network Architectures. ICML 2015.

[22] Jozefowicz, R., Vinyals, O., Schuster, M., Shazeer, N., Wu, Y. Exploring the Limits of Language Modeling. arXiv:1602.02410, 2016.

[23] Kalchbrenner, N., Blunsom, P. Recurrent Continuous Translation Models. EMNLP 2013.

[24] Kaplan, J., et al. Scaling Laws for Neural Language Models. arXiv:2001.08361, 2020.

[25] Khandelwal, U., He, H., Qi, P., Jurafsky, D. Sharp Nearby, Fuzzy Far Away: How Neural Language Models Use Context. ACL 2018; arXiv:1805.04623.

[26] Kneser, R., Ney, H. Improved Backing-off for m-gram Language Modeling. ICASSP 1995.

[27] Levy, O., Goldberg, Y. Neural Word Embedding as Implicit Matrix Factorization. NIPS 2014.

[28] Luong, M.-T., Pham, H., Manning, C. Effective Approaches to Attention-based Neural Machine Translation. EMNLP 2015; arXiv:1508.04025.

[29] Mikolov, T., Karafiát, M., Burget, L., Černocký, J., Khudanpur, S. Recurrent Neural Network Based Language Model. Interspeech 2010.

[30] Mikolov, T., Chen, K., Corrado, G., Dean, J. Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781, 2013. *(2013a)*

[31] Mikolov, T., Sutskever, I., Chen, K., Corrado, G., Dean, J. Distributed Representations of Words and Phrases and their Compositionality. NIPS 2013. *(2013b)*

[32] Papineni, K., Roukos, S., Ward, T., Zhu, W.-J. BLEU: a Method for Automatic Evaluation of Machine Translation. ACL 2002.

[33] Pascanu, R., Mikolov, T., Bengio, Y. On the Difficulty of Training Recurrent Neural Networks. ICML 2013; arXiv:1211.5063.

[34] Rumelhart, D., Hinton, G., Williams, R. Learning Representations by Back-propagating Errors. Nature 323:533–536, 1986.

[35] Sennrich, R., Haddow, B., Birch, A. Neural Machine Translation of Rare Words with Subword Units. ACL 2016; arXiv:1508.07909.

[36] Shannon, C. A Mathematical Theory of Communication. Bell System Technical Journal 27, 1948.

[37] Shannon, C. Prediction and Entropy of Printed English. Bell System Technical Journal 30, 1951.

[38] Shazeer, N., et al. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. ICLR 2017; arXiv:1701.06538.

[39] Shoeybi, M., et al. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053, 2019.

[40] Sutskever, I., Vinyals, O., Le, Q. Sequence to Sequence Learning with Neural Networks. NIPS 2014; arXiv:1409.3215.

[41] Szegedy, C., et al. Rethinking the Inception Architecture for Computer Vision. CVPR 2016 (label smoothing).

[42] Vaswani, A., et al. Attention Is All You Need. NIPS 2017; arXiv:1706.03762.

[43] Werbos, P. Backpropagation Through Time: What It Does and How to Do It. Proc. IEEE 78(10), 1990.